Does INT8 make Leela stronger per second?

Quantizing and pruning a chess transformer, from network outputs to games under a clock

Ritwik Mishra (ME23B237) Anjan R D (ME23B157)
CS6886 Systems for Deep Learning, IIT Madras • https://github.com/anjanrd/CS6886_Group10_CourseProject

Abstract

A faster chess network is only worth having if the extra speed wins games. We take Leela Chess Zero's BT3 transformer, compress it with a sequence of INT8 post-training quantization methods and two 2:4 pruning methods, and follow each version through raw move quality, tree search, timed search and complete games on an RTX 4060 laptop GPU. The best quantized model, which uses SmoothQuant, gets exactly as many endgame positions right as the FP16 original (6,706 of 6,720), runs 40.8% more search visits per second, cuts network latency by 27.9% and uses 26.7% less GPU energy per visit. Search removes the small errors that quantization introduces: every dense INT8 model scores 100% on the main test set from 400 nodes up. In 1,000 games at equal time, the quantized engine is +30 Elo stronger at 100 ms per move (95% CI +15 to +45) and level at 200 ms. Pruning does not get the same treatment from search. Its errors are systematic, and more visits do not fix them.

1 Introduction

Leela Chess Zero (lc0) picks moves by running Monte Carlo tree search (MCTS) guided by a neural network. The network gives two things to the search: a policy, which says which moves look promising, and a value, which says who is winning. Compressing that network has two opposing effects. It adds error to both outputs. It also makes each evaluation cheaper, so the search can look at more positions in the same time.

Which effect wins is an empirical question, and published compression results do not answer it. Accuracy on a held-out set says nothing about what happens once a search sits on top, and a speedup in a benchmark says nothing about games. We therefore measure the whole chain, one link at a time:

Q1 How much does compression change the network's policy, value and first-choice move?

Q2 How much of that change does search undo, and is quantization error different from pruning error in this respect?

Q3 Does the hardware speedup turn into more search, better decisions under a time limit and more won games?

2 Method

2.1 Network and runtime

BT3 (768 wide, 15 layers, 24 heads) reads a 112×8×8 board encoding and outputs a 1858-way move policy, a win/draw/loss value and a moves-left estimate. We export it to ONNX with lc0's own tool, apply each compression method to the ONNX graph, and wrap the result back into a network file that stock lc0 v0.33 loads without changes. All engines run through lc0's TensorRT 10.12 backend. The FP16 baseline is the uncompressed graph run in half precision; compressed models are built with INT8 kernels enabled.

2.2 Compression variants

Table 1 lists the variants. The quantization methods form a ladder in which each step adds one technique, so the effect of each can be read off directly. All variants use signed symmetric INT8 for weights and activations (W8A8) and share one calibration set of 512 positions, half from random games and half from endgames. The two free settings, the clipping percentile and the SmoothQuant strength α, were picked on a separate 3,360-position development set and kept fixed for every test reported here. Percentile 99.999 beat 99.9, 99.99 and entropy calibration, and α=0.7 was best among values from 0.3 to 0.8.

Table 1: Variants tested. Quantization steps are cumulative.

Name	What it adds
FP16	uncompressed baseline
per-tensor	one INT8 scale per tensor, min/max range
per-channel	one weight scale per output channel
+clip	activations clipped at the 99.999th percentile
INT8-SQ	SmoothQuant, α=0.7 **(main model)**
SQ+attn	also quantizes attention score and value products
magnitude 2:4	50% magnitude pruning, then INT8-SQ
SparseGPT 2:4	50% SparseGPT pruning, then INT8-SQ
policy-/value-only	INT8 in one output head, FP16 in the other

SmoothQuant [2] moves activation outliers into the weights by rewriting XW as $(X/s)(sW)$. In BT3 the normalized activations feed both the next layer and a residual connection, so we apply $1/s$ as an explicit multiply in front of the quantized layer instead of folding it into the normalization. For pruning we compare plain magnitude pruning with SparseGPT [3], which updates the remaining weights to compensate for the ones it removes. Both prune to the 2:4 pattern that NVIDIA GPUs can accelerate [4] and then apply the INT8-SQ recipe.

2.3 Test positions

Positions with five or fewer pieces have known perfect answers in Syzygy tablebases, which gives an exact referee for every move [1]. A move counts as correct if it keeps the position's game-theoretic result (a win stays a win, a draw stays a draw). The engines themselves never see the tablebases.

We use four sets. *Main* has 6,720 positions with 3 to 5 pieces, 240 from each of 28 material classes, all chosen so that the choice of move matters. *Fresh* has 24,000 new five-piece positions, with mirrored and rotated duplicates of anything in Main removed. Because FP16 already gets nearly all of these right, we also build two hard subsets using a rule that looks only at the FP16 model: positions where its first-choice move is wrong or where it puts less than 80% of its policy on correct moves. That gives *Hard-Main* (99 positions) and *Hard-Fresh* (632). Differences between models are paired by position, and confidence intervals come from 10,000 bootstrap resamples stratified by material class.

2.4 Evaluation stages

Each variant passes through the same five stages.

S1 Raw network. One node of search, so the move is the network's own first choice. We also record policy KL diver-

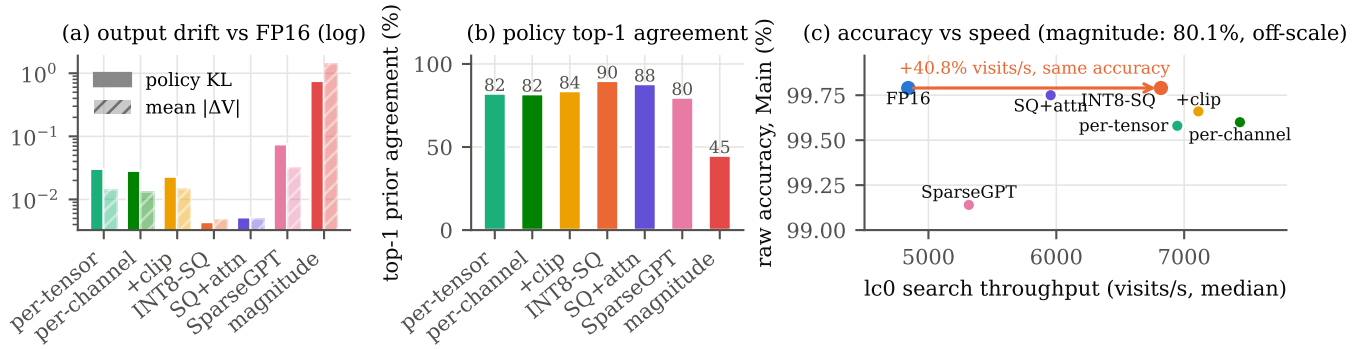


Figure 1: Network-level effects on Main. (a) Mean policy KL divergence and value change against FP16 (log scale). (b) How often the top policy move matches FP16's. (c) Raw accuracy against measured search speed. INT8-SQ has FP16's accuracy and 40.8% more visits per second.



Figure 2: Search on the hard subsets. (a, b) Accuracy against nodes per move (raw = one node). (c) Accuracy gap to FP16 on Hard-Fresh. Every dense INT8 model closes in on FP16; SparseGPT stays about 19 points behind.

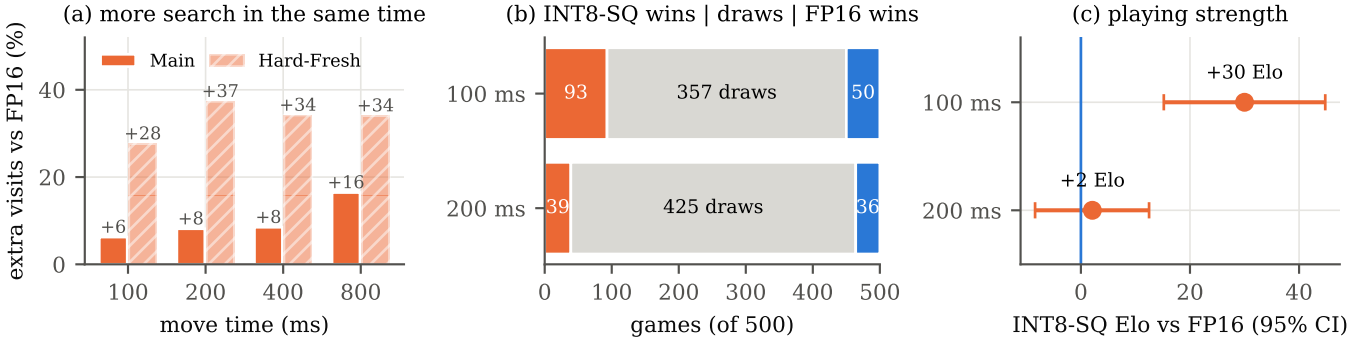


Figure 3: Equal-time comparison. (a) Extra visits INT8-SQ completes in the same move time. (b) Results of 500 games per time control. (c) Elo difference with 95% intervals, computed from game pairs that share an opening.

gence and value change against FP16.

S2 Fixed-node search. 10 to 1,600 nodes per move, so every model gets the same amount of search regardless of speed.

S3 Fixed-time search. 100, 200, 400 and 800 ms per move, so faster models get more nodes.

S4 Games. INT8-SQ against FP16 at 100 and 200 ms per move: 250 balanced six-move openings [6], each played once with each colour, 500 games per time control. Games are adjudicated by tablebase at five pieces and by standard resign and draw rules.

S5 Hardware. Repeated lc0 benchmarks for search speed, network latency at several batch sizes, NVML energy counters, TensorRT layer profiles and a roofline [5] against measured peak rates.

3 Results

3.1 Q1: what INT8 does to the network

Each step of the ladder does what it is meant to (Figure 1). The simplest INT8 model, per-tensor, shifts the policy (KL 0.030) and value (0.014) enough to cost 0.21 points of raw accuracy, with 15 positions lost and one gained. Per-channel scales and clipping give modest improvements. SmoothQuant removes most of what remains: policy KL drops to 0.0044, seven times lower than per-tensor, value change drops to 0.0048, and the top policy move matches FP16's 89.9% of the time.

The result is that INT8-SQ gets exactly the same number of Main positions right as FP16, 6,706 of 6,720. It wins two positions FP16 misses and misses two that FP16 wins. The reason the earlier steps struggle is visible in the activations. In one representative layer the largest channel peaks at 53.5 before smoothing and 2.27 after, and 100 of the 101 smoothed layer groups end up with a flatter activation profile.

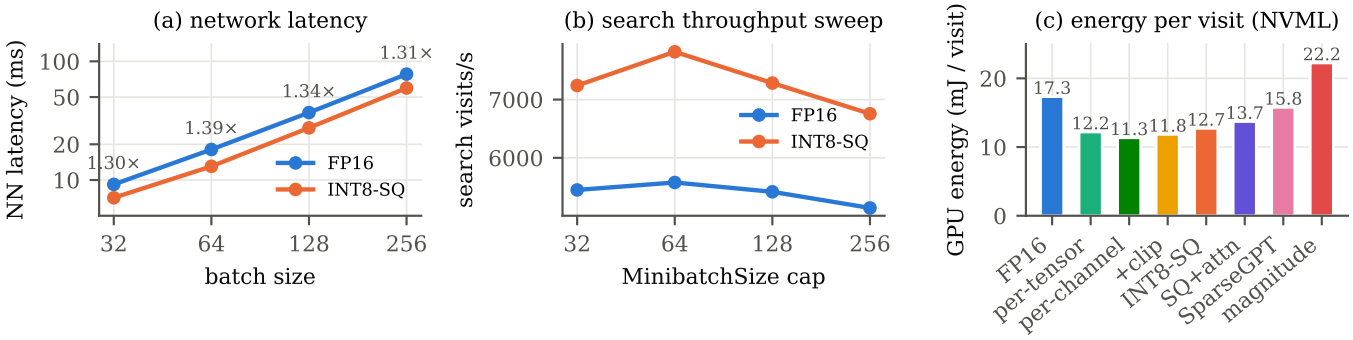


Figure 4: Hardware measurements on an RTX 4060 laptop GPU. (a) Network latency per batch with FP16/INT8-SQ ratios. (b) Search speed at four lc0 batch-size caps. (c) Median GPU energy per search visit.

Quantizing the attention products as well (SQ+attn) costs almost nothing: 6,703 on Main, and the best INT8 score on Hard-Fresh at 553 of 632, two behind FP16. The single-head models show where the remaining raw error lives. With only the value head in INT8, raw accuracy is identical to FP16 on every set. With only the policy head in INT8, it matches INT8-SQ. The leftover error is in the policy.

Pruning is much rougher. Magnitude 2:4 drops raw accuracy to 80.1% and pushes policy KL to 0.75. SparseGPT, at the same 50% sparsity, keeps 99.1% and cuts policy KL tenfold, to 0.075. Weight compensation is clearly worth doing.

3.2 Q2: search repairs quantization, not pruning

On Main the answer comes quickly. At 100 nodes FP16 and INT8-SQ both score 6,718 of 6,720. From 400 nodes up, every dense INT8 model, including both single-head versions, scores all 6,720.

The hard subsets show how it happens (Figure 2). Per-tensor INT8 starts 11.1 points behind FP16 on Hard-Main and 7.3 behind on Hard-Fresh, and the gap shrinks with every increase in nodes. By 200 nodes it is about one point on both sets. INT8-SQ starts closer and finishes within 0.3 points of FP16 on Hard-Fresh (626 against 628). SQ+attn finishes one position ahead (629). Quantization error is small and scattered, so when the search looks at a few more continuations, errors in different directions cancel.

Pruned models behave differently. SparseGPT improves with search in absolute terms, from 74% to 81% on Hard-Fresh, but stays about 19 points behind FP16 at every budget. Magnitude pruning stays at 55 to 58% on Main even with 400 nodes. Removing half the weights produces errors that point the same way every time, and searching more with the same flawed evaluator does not correct them. At the same weight-storage saving, quantization is far kinder to playing quality than pruning, and search hides its damage far better.

The same picture shows up in node equivalents. Per-channel, +clip and SQ+attn at 100 nodes each match FP16 run with about 320 nodes, and INT8-SQ matches FP16 at each budget we tested.

3.3 Q3: speed, timed search and games

Speed. Median search speed is 4,842 visits per second for FP16 and 6,818 for INT8-SQ, a 40.8% increase. Network latency at batch 64 falls from 18.1 to 13.0 ms, and the speedup holds at 1.31 to 1.39× across batch sizes 32 to 256 (Figure 4a). TensorRT's layer report shows all 164 weight matrix multiplies running as INT8 GEMMs, which covers 97.9% of the network's 9.1 billion matrix operations per position.

Timed search. With the same time per move, INT8-SQ completes 6 to 16% more visits on Main and 28 to 37% more on Hard-Fresh (Figure 3a). The gain is larger on hard positions because the search runs longer there before stopping early. Main accuracy is already at its ceiling, and INT8-SQ is never behind FP16 at any time limit (6,718, 6,720, 6,720 and 6,720 against 6,718, 6,719, 6,720 and 6,720). On Hard-Fresh the two stay within one position of each other throughout, with INT8-SQ ahead at 800 ms.

Table 2: FP16 against INT8-SQ along the whole chain.

Measure	FP16	INT8-SQ	Change
Raw accuracy, Main (of 6,720)	6,706	6,706	±0
Accuracy at 400 nodes, Main	6,720	6,720	±0
Network latency, batch 64 (ms)	18.08	13.04	−27.9%
Search speed (visits/s)	4,842	6,818	+40.8%
GPU energy (mJ per visit)	17.34	12.72	−26.7%
Visits at 800 ms, Hard-Fresh	2,428	3,255	+34.1%
Wins at 100 ms (500 games)	50	93	+30 Elo
Wins at 200 ms (500 games)	36	39	+2 Elo

Games. At 100 ms per move INT8-SQ won 93 games and lost 50, with 357 draws. That is a score of 54.3%, or +30 Elo with a 95% interval of +15 to +45 ($z = 4.0$). At 200 ms the result was 39 wins to 36 with 425 draws, which is +2 Elo (−8 to +13).

The gap narrows with more time, which is what one would expect. At 100 ms each engine searches a few hundred to a few thousand nodes per move, and 40% more of them buys real extra depth. At 200 ms the draw rate rises to 85% and each additional node is worth less, the usual diminishing return of search time in chess engines. Across both time controls INT8 never made the engine weaker, and it made it clearly stronger where time was short.

3.4 Hardware and energy

GPU energy per visit falls from 17.3 to 12.7 mJ with INT8-SQ, and every dense INT8 model lands between 11.3 and 13.7 mJ (Figure 4c). The energy counter and sampled power readings agree to within 0.6% under load. TensorRT profiles at batch 64 show engine time falling from 16.2 to 12.1 ms, with most of the saving in the matrix multiplies. Normalization, softmax and precision conversions make up a larger share of what is left, which is where further speedups would have to come from. On a roofline built from measured peaks (48.9 FP16 TFLOP/s, 108.7 INT8 TOP/s, 249 GB/s) the large BT3 matrix multiplies sit near the compute limit rather than the memory limit, which is why halving the operand width pays off.

Search speed peaks at a batch cap of 64 for both models, where INT8-SQ reaches 7,820 visits per second against FP16's 5,577 (Figure 4b). The INT8 advantage holds at every cap.

For the pruned models, TensorRT picks 2:4 sparse kernels for 126 layers of the magnitude model and 125 of the SparseGPT model, which makes the engine a further 7.7% and 6.6% faster at batch 256. lc0 does not currently ask TensorRT for sparse kernels, so this gain does not reach the search yet. Inside lc0 today the pruned models are actually slower to search (3,775 and 5,316 visits per second) even though each network call is fast, a reminder that network latency alone does not set search speed.

The per-channel model is the fastest of all at 7,436 visits per second. We still treat INT8-SQ as the main model because it was chosen on the development set for quality, and Figure 1c shows it gives the best accuracy for almost the same speed.

4 Limitations

The study covers one network on one laptop GPU running at its natural clock speeds. The test positions are endgames, generated rather than taken from games, and the main set is close to its ceiling. The hard subsets are defined by the FP16 model, which makes them slightly favourable to FP16 by construction. lc0 prints policy values with limited precision, which caps KL resolution near 10^{-4}. The games use one opening book and two short time controls, and energy covers the GPU only. Quantization-aware training, distillation, other network sizes and alpha-beta search were outside this study.

All models, test positions, per-position results, game records and hardware logs are released with a single reproduction script. Rebuilding the compressed models from the original checkpoint gives bit-identical graphs and CPU outputs. Freshly built TensorRT engines can differ slightly in FP16 output on 2 of 32 test positions (largest value change 0.011), because TensorRT chooses kernels per build; whether each move was correct did not change.

5 Conclusion

For BT3, INT8 post-training quantization costs almost nothing in move quality and gains a lot in speed. With SmoothQuant the network keeps FP16's raw accuracy, search removes the remaining error within a few hundred nodes, and the engine runs 41% more visits per second on 27% less energy per visit. Under an equal clock this gave $+30$ Elo at 100 ms per move and no loss at 200 ms. Pruning is a different case. Its errors are systematic, so search cannot average them away, although SparseGPT shrinks them by an order of magnitude compared with magnitude pruning. Two follow-ups look worthwhile: letting lc0 use TensorRT's sparse kernels, and checking how the Elo gain changes at longer time controls and on other network sizes.

6 Prior work

We build on Leela Chess Zero (lc0), the open-source neural-network chess engine. Its BT3 network, ONNX export tools and search/runtime underpin our quantization, pruning and evaluation study.
https://github.com/LeelaChessZero/lc0

References

[1] R. Haque, T.-H. Wei, M. Müller. On the road to perfection? Evaluating Leela Chess Zero against endgame tablebases. *ACG 2021*, LNCS 13262, 2022.

[2] G. Xiao et al. SmoothQuant: accurate and efficient post-training quantization for large language models. *ICML*, 2023.

[3] E. Frantar, D. Alistarh. SparseGPT: massive language models can be accurately pruned in one-shot. *ICML*, 2023.

[4] A. Mishra et al. Accelerating sparse deep neural networks. arXiv:2104.08378, 2021.

[5] S. Williams, A. Waterman, D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *CACM*, 2009.

[6] J. H. Thomas. Balanced six-ply opening books derived from Caissabase. lczero-training wiki.